

Restaurant SaaS AI Agent Handoff Report

Date: 2026-02-28

Repo: <https://github.com/RaheemNadeem/restaurant-saas-platform>

1) GitHub Sync + PR Merge Status

- Remote synced with 'origin' successfully ('git pull --ff-only').
- Open PRs checked via GitHub CLI: * * none open * * .
- Result: no pending PRs to merge at this time.

2) What Changed Recently (Branch/Commit Intelligence)

Main branch latest

- 'cb218d9' feat(ui): apply playful goofy visual style to preview screens (#24)
- '86401f9' feat(frontend): apply mobile responsive baseline and standards (#22)
- '3fa7a9a' feat(frontend): add login page UI preview (#20)
- '6372b44' feat(devops): add github pages deployment for online ui preview (#18)

Active feature branch ('feat/23-goofy-visual-style')

Additional commits on top of main include:

- '543ac14' backend tenant-isolation baseline + demoshell
- '8f492c9' OpenAPI draft v1 for tenant-aware menu endpoints
- '9fffee6' weekly status template + reporting cadence
- '0432229' dailysprint plan (2026-02-22)

New remote branch observed

- 'origin/codex/demo-mode-checkout-updates'
- '9832247' Add demo-mode UI flow updates and session context
- 'dfd40e1' Update session context with latest progress

3) Current Architecture Snapshot (Single-Page Understanding)

Stack

- Frontend: Angular
- Backend: .NET8 Web API
- Database: PostgreSQL
- Realtime: SignalR
- Payments: Stripe Connect

Monorepo layout

- 'frontend/' Angular apps/libs and UI preview
- 'backend/' API modules and integration tests
- 'infra/' Postgres baseline SQL + infra assets
- 'docs/' PRDs, API contracts, workflow, standards, status docs

Tenant isolation model (baseline)

- Tenant context propagated via 'X-Tenant-Id'
- Tenant-scoped menu endpoints in API contract
- RLS baseline SQL defined at 'infra/postgres/001_tenant_rls_baseline.sql'
- Integration test evidence exists under 'backend/tests/Api.Integration Tests'

API contract baseline (Sprint 1)

OpenAPI draft ('docs/api/openapi-v1.yaml') includes:

- 'GET /health'
- 'GET /api/menu/items'
- 'POST /api/menu/items'
- 'PUT /api/menu/items/{id}'

(All tenant-scoped endpoints require 'X-Tenant-Id'.)

4) UI Pages + Flow Map (for Any AI Agent)

Implemented preview/shell pages

- Auth ('/auth')
- Onboarding ('/onboarding')
- Dashboard ('/dashboard')
- Menu ('/menu')
- Plus:skeletons for Storefront, Checkout, Merchant Admin, Staff Board

Primary user flows

1. Merchant onboarding flow:

'/auth' -> '/onboarding' -> '/dashboard'

2. Menu management flow:

'/dashboard' -> '/menu' -> tenant-scoped API calls

3. Demo/verification flow:

frontend route walkthrough + backend tenant-separation checks

Responsive acceptance baseline

- Breakpoints: mobile/tablet/desktop
- Mandatory checks at widths: 320, 375, 390, 768, 1024, 1440
- No horizontal overflow at 320px
- Minimum touch target: 44x44

5) Sprint Issues Ledger (Current)

Open GitHub issues discovered:

- #2 Implement tenant isolation baseline (tenant_id + PostgreSQL RLS)
- #3 Angular shell architecture (apps/libs + routing)
- #4 Stripe Connect onboarding + payment intent flow
- #5 Order lifecycle state machine + SignalR updates
- #9 PostgreSQL schema baseline + tenant_id conventions
- #10 Enforce PostgreSQL Row-Level Security (RLS)
- #15 UI foundation: theme system + preview screens

6) Recommended Sprint Breakdown (Documentation-First)

This section is written so another AI agent can pick up execution with minimal ambiguity.

Sprint A: Foundation Closure (1 week)

1. Finalize DB-backed tenant isolation path (close #2/#9/#10 dependencies)
2. Replace in-memory menu repository with EF Core + Postgres
3. Add migration + seed strategy by tenant
4. Expand integration tests: cross-tenant read/write denial

*** DoD: ***

- RLS enforced in runtime DB
- Integration tests prove tenant leakage = 0
- API docs updated to match implemented behavior

Sprint B: Merchant Core (1 week)

1. Merchant onboarding end-to-end happy path
2. Menu CRUD complete with validation and error contracts
3. Merchant dashboard metrics placeholders wired from API

*** DoD: ***

- Merchant can create account/store, create menu items, publish state

Sprint C: Checkout + Payments (1-2 weeks)

1. Public storefront to checkout flow
2. Stripe Connect onboarding + PaymentIntent flow (issue #4)
3. Payment failure/retry/idempotency handling

*** DoD: ***

- First paid order completed in sandbox end-to-end

Sprint D: Ops + Realtime (1 week)

1. Staff order board live updates via SignalR (#5)
2. Order state machine + transition guards
3. Audit/event logging for order transitions

*** DoD: ***

- Staff sees live order updates, controlled transitions, and recovery behavior

7) AI Agent Execution Checklist (Copy/Paste)

1. `git fetch --all --prune`
2. Verify branch target (`main` for release work)
3. Read in order:
 - `README.md`
 - `docs/prds/SPRINT-1-PLAN.md`
 - `docs/prds/PRD-001-merchant-onboarding-and-storefront-mvp.md`
 - `docs/api/openapi-v1.yaml`
 - `docs/RESPONSIVE\_STANDARDS.md`
4. Validate unresolved issue scope (#2, #3, #4, #5, #9, #10, #15)
5. Implement smallest vertical slice with tests
6. Open PR linked to issue, include demo evidence and risk notes

8) Immediate Next Actions (High Value)

1. Merge `feat/23-goofy-visual-style` strategy into `main` via PR (if still intended)
2. Review `origin/codex/demo-mode-checkout-updates` and decide merge/discard
3. Prioritize closing #9 + #10 before expanding checkout/payment complexity

This report is designed as a single-source handoff so any new AI agent can quickly understand architecture, flow, issue status, and sprint priorities.